

Documentação de Arquitetura

Versão do app 1.1.0 · Onboarding técnico · Julho 2026

Sumário

- 1. Visão geral
- 2. Stack tecnológica
- 3. Estrutura de pastas
- 4. Arquitetura Electron
- 5. Fluxo de abertura
- 6. Autenticação e perfis
- 7. Banco de dados
- 8. Cadastro de associados
- 9. Regras de negócio
- 10. Integração Intelbras XPE
- 11. Módulos da interface
- 12. Como adicionar um novo módulo
- 13. Limitações e pendências
- 14. Referência rápida da API

1. Visão geral

O **AquaAccess Facial** é uma aplicação desktop **offline-first** para **controle de acesso às piscinas** de um clube. O fluxo principal cobre:

- 1. Cadastro de associados (chamados de *pacientes* na UI e no banco)
- 2. Registro e validade de exames médicos
- 3. Liberação ou negação de acesso à catraca/piscina
- 4. Integração em rede com o videoporteiro **Intelbras XPE 3200 PLUS IP**

Não há backend remoto obrigatório: tudo roda localmente no PC do clube. A tela de sincronização com nuvem (Supabase) existe na UI, mas ainda é **placeholder**.

2. Stack tecnológica

Camada	Tecnologia
Runtime desktop	Electron 41
Interface	HTML, CSS e JavaScript vanilla
Banco local	sql.js (SQLite em WASM)
Autenticação	bcryptjs
Integração XPE	HTTP local + TCP + Playwright (opcional)
Empacotamento	electron-builder (NSIS Windows)

Comando	Função
<code>npm start</code>	App em desenvolvimento
<code>npm run dist</code>	Instalador em <code>release/</code>
<code>npm run xpe:install-browser</code>	Instala Chromium do Playwright
<code>npm run xpe:test</code>	Teste isolado da automação XPE

3. Estrutura de pastas

```
facial/
├─ main.js           # Processo principal (janela, IPC, bridge)
├─ preload.js       # contextBridge → window.clubAccess
├─ db.js            # Schema SQLite + regras de negócio
├─ xpeBridge.js      # Servidor HTTP para eventos do XPE
├─ xpeSetup.js       # Descoberta de IP/LAN e assistente
├─ xpeAutomation/   # Playwright (painel web do XPE)
├─ renderer/
│   ├─ login/        # Tela de login
│   ├─ medico/       # Painel médico
│   ├─ admin/        # Shell admin + views
│   └─ shared/       # CSS compartilhado
├─ docs/            # Documentação
└─ release/         # Artefatos de build
```

Dados em runtime (Windows)

Pasta típica: %APPDATA%\AquaAccess Facial\

Item	Conteúdo
club-piscinas.db	Banco SQLite
patient-photos/	Fotos dos associados
device-settings.json	Configurações gerais
xpe-bridge-settings.json	Bridge HTTP / OpenDoor
xpe-integration.json	Estado da integração XPE
xpe-export/	Exportação de usuários
xpe-bridge-inbound.ndjson	Log bruto do XPE

4. Arquitetura Electron

```
Processo Principal (main.js)
├─ db.js           → domínio / SQLite
├─ xpeBridge.js    → HTTP ← XPE
├─ xpeSetup.js     → descoberta / assistente
├─ xpeAutomation/ → Playwright → painel web XPE

preload.js → window.clubAccess.*

Renderer: login → medico | admin
```

Segurança: contextIsolation: true , nodeIntegration: false . Comunicação apenas via IPC.

Padrão para feature nova: db.js → main.js (IPC) → preload.js → renderer.

5. Fluxo desde a abertura

1. Usuário executa npm start ou o instalador.
2. app.whenReady : initDb() , auto-discover XPE, bridge HTTP.
3. createWindow() abre renderer/login/login.html .

4. Login via `window.clubAccess.login` → `authenticateUser` .
5. Sessão em `sessionStorage` (`clubAccessRole` , usuário, nome).
6. Redireciona: admin → `admin/home.html` ; médico → `medico/medico.html` .

Credenciais padrão (seed)

Perfil	Usuário	Senha
Administrador	admin	admin
Médico	medico	admin

6. Autenticação e perfis

- Hash bcrypt; checagem de usuário ativo.
- Roles: apenas **admin** ou **medico**.
- Admin pode entrar na aba médico; médico não entra como admin.
- Não há JWT — sessão só no renderer.

Funções de CRUD de usuários do sistema existem em `db.js` (`createSystemUser` , etc.), mas ainda **não estão no preload/IPC/UI**.

Atenção: hoje não há autorização por role no IPC. Qualquer renderer autenticado pode chamar as APIs do preload.

7. Banco de dados

Engine **sql.js**. Cada escrita chama `persist()` e grava `club-piscinas.db` .

Tabela	Conteúdo
users	Operadores do software (admin/médico)
patients	Associados: nome, CPF, telefone, foto, blocked
exams	Exames com validade e status
access_events	Histórico liberado/negado

O `patients.id` é o mesmo número usado como **ID Usuário** no XPE.

8. Cadastro de associados

Fluxo principal no painel médico:

1. Nome, telefone (obrigatório), CPF (opcional).
2. Foto obrigatória (arquivo ou câmera).
3. `patientsCreate` → `createPatient` → grava foto em `patient-photos/`.

No domínio: associados do clube = `patients` ; operadores do app = `users` .

9. Regras de negócio

Centro de verdade: `db.js` .

Liberação — `registerPoolAccess`

Concede acesso se:

1. Paciente existe
2. Não está `blocked`
3. Tem exame `Válido` com `valid_until` ≥ hoje (inclui “vence nesta semana”)

Caso contrário, nega. Sempre grava em `access_events` .

Exame — `createExam`

- Validade = data do exame + N dias (configurável; padrão 30)
- Alinha a dias da semana permitidos
- Exames vigentes anteriores → `Substituído` ; novo → `Válido`

10. Integração Intelbras XPE

1. XPE envia evento HTTP para o PC (ex.: porta `37891` , path `/intelbras/xpe`).
2. Bridge extrai `userId` ou CPF.
3. `findPatientIdForXpeBridge` resolve o paciente.
4. Chama a mesma `registerPoolAccess` .
5. Se liberado e OpenDoor ativo → HTTP OpenDoor no equipamento.

Exportação de fotos/CSV e sync Playwright ficam em Dispositivo facial (admin). Ajuste de seletores: `xpeAutomation/selectors.js` .

11. Módulos da interface

Médico

- Pacientes — cadastro, foto, visão por atenção
- Exames — validação e histórico

Admin

View	Função
Visão geral	Dashboard e status do leitor
Pacientes	Lista, edição, bloqueio, sync
Controle de acessos	Eventos e stats do dia
Dispositivo facial	TCP, export, bridge, Playwright
Relatórios	PDF / CSV
Sincronização	Local OK; nuvem placeholder
Exames	Gestão administrativa
Configurações	Clube, regras, XPE, bridge

12. Como adicionar um novo módulo

1. Funções em `db.js` e export.
2. `ipcMain.handle` em `main.js`.
3. Expor em `preload.js` (`window.clubAccess`).
4. Admin: view HTML + `VIEW_FILES` + nav + lógica em `admin.js`.
5. Médico: seções em `medico.html` + `medico.js`.
6. Integração externa: módulo no main (como `xpeBridge.js`).

Não acessar Node/filesystem/banco direto no renderer.

13. Limitações e pendências

Item	Situação
Sync nuvem / Supabase	UI placeholder
CRUD de usuários do sistema	Só no <code>db.js</code> , sem IPC/UI
Autorização por role no IPC	Não implementada
Browsers Playwright no .exe	Instalar no PC destino
<code>admin.js</code>	Monólito grande (~2.5k linhas)

14. Referência rápida da API (preload)

Método	Uso
<code>login</code>	Autenticação
<code>patientsCreate</code> / Update / List / ...	Pacientes
<code>examsRegister</code> / List / Delete	Exames
<code>accessRegisterPool</code> / List / Stats	Acessos
<code>reportsSummary</code> / Rows	Relatórios
<code>dashboardSnapshot</code>	Dashboard
<code>deviceSettingsGet</code> / Set	Configurações
<code>xpeBridge*</code> / <code>xpeExport*</code> / <code>xpeSyncUser</code>	Integração XPE
<code>xpeSetupDiscover</code> / Apply	Assistente Intelbras